

ncurses

Création d'une interface avec *ncurses*

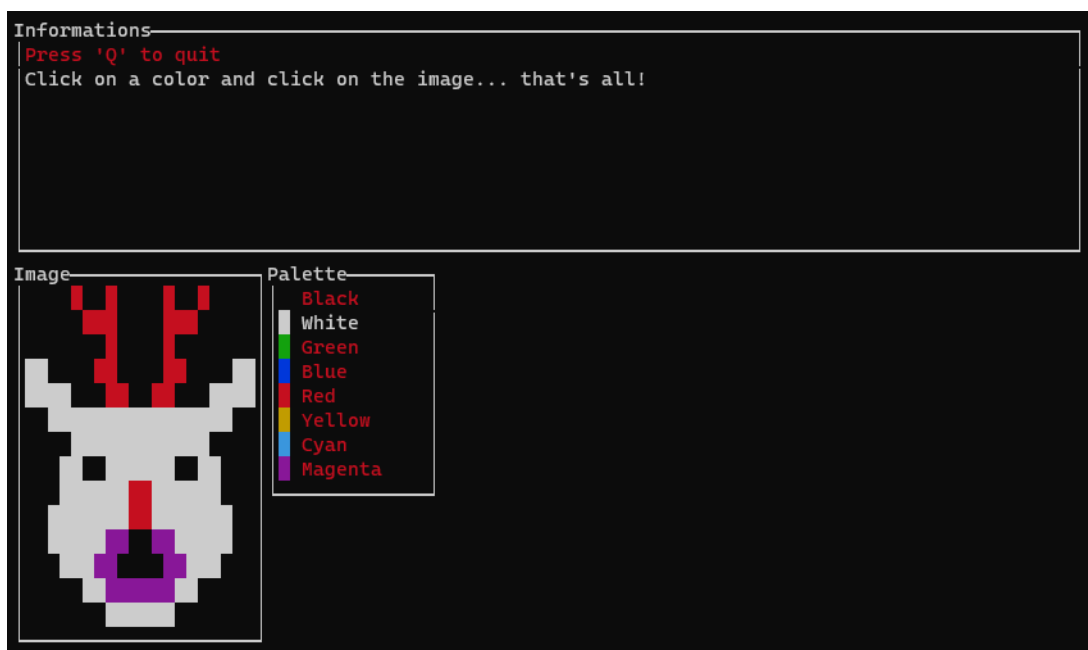
Ce tutoriel présente un programme qui exploite une interface *ncurses*. Il s'agit d'un éditeur d'image qui peut créer ou modifier des images stockées dans un fichier binaire.



Il est possible d'exploiter tout ou une partie du code dans les projets.

1 Présentation de l'application

L'application permet de dessiner une image en utilisant huit couleurs de base. L'utilisateur en sélectionne une à l'aide de la souris (en cliquant simplement dessus dans la fenêtre «Palette») puis clique à l'endroit choisi dans l'image. Un carré de la couleur choisie est alors affiché.



Les différentes parties du code sont expliquées dans les sections suivantes.

2 Gestion des fenêtres : **window.h** et **window.c**

Avec *ncurses*, il est possible de créer des cadres aux fenêtres (fonction *ncurses* box). Cependant, il est intégré à l'intérieur de la fenêtre : les prochaines écritures ou l'effacement du contenu de la fenêtre vont donc le recouvrir. Une solution présentée en cours est d'exploiter une fenêtre dans laquelle on crée le cadre et une sous-fenêtre dans laquelle on écrit. Dans le fichier `window.h`, nous définissons la structure suivante :

```
typedef struct {  
    int posx, posy;           // Window position  
    int width, height;       // Window dimensions  
    WINDOW *box;             // Box of the window  
    WINDOW *inner;           // The innner window  
} window_t;
```

La fonction `window_create` permet d'allouer et d'initialiser une structure `window_t`, créer les fenêtres associées en ajoutant un cadre et un titre. Il faut spécifier la position (*posX*, *posY*) et les dimensions (*width*, *height*). Il est également possible d'indiquer si on active le défilement avec le dernier paramètre (si le défilement n'est pas actif et que l'on écrit plus de lignes que la hauteur de la fenêtre intérieure, les caractères ne sont pas affichés).

```
window_t *window_create(int posX, int posY, int width, int height, char  
    *title, bool scroll);
```

Lorsque cette méthode est appelée, une structure `window_t` est allouée. Pour la supprimer de la mémoire proprement, il faut appeler la méthode `window_delete`.

```
void window_delete(window_t **window);
```

Les deux fonctions `fenetre_isin` et `window_getcoordinates` permettent respectivement de vérifier si une position donnée est présente dans la fenêtre et de récupérer les coordonnées au sein de la fenêtre, le cas échéant.

```
bool window_isin(window_t *window, int x, int y);  
  
bool window_getcoordinates(window_t *window, int posx, int posy, int *x,  
    int *y);
```



À la place de ces fonctions, il est possible d'utiliser la fonction `ncurses wmouse_trafo`.

L'écriture au sein d'une fenêtre telle que définie par la structure `window_t` implique d'écrire dans la fenêtre `ncurses` intérieure. Pour simplifier le code, il est possible d'utiliser les fonctions suivantes pour afficher des caractères :

```
// Affiche un caractère dans la fenêtre.
int window_addch(window_t *window, chtype c);

// Affiche un caractère dans une fenêtre à une position donnée.
int window_mvaddch(window_t *window, int posY, int posX, chtype c);

// Affiche un caractère dans une fenêtre dans une couleur donnée.
int window_addch_col(window_t *window, unsigned int color, chtype c);

// Affiche un caractère dans une fenêtre à une position et une couleur
// donnée.
int window_mvaddch_col(window_t *window, int posY, int posX, unsigned
    int color, chtype c);
```

De même, les fonctions suivantes permettent d'afficher des chaînes de caractères (en utilisant le format classique de printf) :

```
// Affiche une chaîne de caractères dans une fenêtre.
int window_printw(window_t *window, const char *fmt, ...);

// Affiche une chaîne de caractères dans une fenêtre dans une couleur
// donnée.
int window_printw_col(window_t *window, unsigned int couleur, const char
    *fmt, ...);

// Affiche une chaîne de caractères dans une fenêtre à une position donn
// ée.
int window_mvprintw(window_t *window, int posY, int posX, const char *
    fmt, ...);

// Affiche une chaîne de caractères dans une fenêtre à une position et
// dans une couleur données.
int window_mvprintw_col(window_t *window, int posY, int posX, unsigned
    int color, const char *fmt, ...);
```

Par défaut, toutes les fonctions d'affichage ne rafraichissent pas la fenêtre. Il faut utiliser la fonction *ncurses* `wrefresh` ou bien la fonction suivante :

```
int window_refresh(window_t *window);
```

Il est possible également de changer la couleur courante avec la fonction suivante :

```
int window_color(window_t *window, unsigned int color);
```

Enfin, il est possible d'effacer le contenu de la fenêtre avec la fonction suivante :

```
int window_erase(window_t *window);
```

3 Gestion des images : **image.h** et **image.c**

Le but de l'application étant de créer une image, nous définissons une structure `image_t` comme suit :

```
typedef struct {
    unsigned short width;           // La largeur
    unsigned short height;          // La hauteur
    unsigned char *image;           // L'image
} image_t;
```

Nous proposons uniquement deux fonctions pour créer une image et pour en supprimer une de la mémoire.

```
// Création d'une image.
image_t *image_create(unsigned short width, unsigned short height);

// Suppression d'une image.
void image_delete(image_t **image);
```

Les deux fonctions suivantes permettent de lire et de sauvegarder l'image dans un fichier binaire.

```
// Sauvegarde d'une image dans un fichier.
void image_save(char *filename, image_t *image);

// Chargement d'une image dans un fichier.
image_t *image_load(char *filename);
```

4 Fonctions *ncurses* : **functions.h** et **functions.c**

Ces fonctions ont été présentées en cours. Elles sont utilisées pour initialiser *ncurses* avec les options par défaut ou pour l'arrêter, pour initialiser les couleurs ou la souris et pour récupérer les coordonnées de la souris lorsqu'un évènement `KEY_MOUSE` est détecté.

5 L'interface : **interface.h** et **interface.c**

Pour manipuler l'interface de l'application, nous avons défini la structure suivante :

```
typedef struct {
    window_t *win_infos; // La fenêtre d'informations
    window_t *win_palette; // La fenêtre de la palette
    window_t *win_image; // La fenêtre de la bitmap
    image_t *image; // L'image
    unsigned int selection; // La couleur sélectionnée
} interface_t;
```

L'interface est constituée de trois fenêtres. La fenêtre «Informations» (`win_infos`) permet d'afficher des informations pour l'utilisateur. Elle peut servir pour le débogage. La fenêtre «Image» (`win_image`) représente l'image que l'utilisateur modifie. La fenêtre «Palette» (`win_palette`)

représente les couleurs possibles. Elle permet à l'utilisateur d'en sélectionner une avec la souris. Le champ `image` correspond à l'image qui est dessinée et le champ `selection` permet de mémoriser la couleur sélectionnée.

Pour créer ou supprimer l'interface, il faut utiliser les fonctions dont les signatures sont ci-dessous. La création de l'interface crée les différentes fenêtres, crée et initialise une image puis affiche le texte par défaut.

```
interface_t *interface_create(image_t *image);

void interface_delete(interface_t **interface);
```

Pour interagir avec l'interface, nous utilisons la fonction `interface_main` qui prend en paramètre l'interface et le caractère saisi. Elle détermine où l'utilisateur a cliqué avec la souris ou quelle touche il a pressée. Elle appelle ensuite l'une des méthodes correspondant à chaque fenêtre.

```
void interface_main(interface_t *interface, int c);
```

La fonction `interface_palette_actions` détermine sur quelle ligne l'utilisateur a cliqué et récupère la couleur correspondante. L'affichage de la fenêtre de la palette est mis à jour pour identifier la couleur courante (le texte est affiché en blanc alors que le texte des autres couleurs est affiché en rouge) grâce à la fonction `interface_palette_update`.

```
// Gestion des actions dans la fenêtre palette.
void interface_palette_actions(interface_t *interface, int posX, int
    posY);

// Met à jour la fenêtre 'palette'.
void interface_palette_update(interface_t *interface);
```

La fonction `interface_image_actions` modifie la case de l'image en fonction de la couleur sélectionnée et met à jour l'affichage. Un carré de la couleur correspondante est affiché à la position cliquée.

```
void interface_image(interface_t *interface, int posX, int posY);
```

6 Le programme principal : **editor.c**

Ce fichier contient uniquement le `main`. Dans un premier temps, il vérifie les arguments. Pour charger une image existante, on spécifie son nom en argument :

```
./editor exemple.bin
```

Pour créer une nouvelle image, on spécifie le nom du fichier, ainsi que la largeur et la hauteur :

```
./editor exemple.bin 40 20
```

Le programme initialise ensuite *ncurses* puis crée l'interface (les dimensions de l'interface sont vérifiées en fonction des dimensions de l'image). La boucle principale est la suivante :

```
while(quit == FALSE) {
    ch = getch();
    if((ch == 'Q') || (ch == 'q'))
        quit = true;
    else
        interface_actions(interface, ch);
}
```

Les touches pressées et la souris sont gérées dans la fonction `interface_actions`. Lorsque la touche Q est pressée, le programme s'arrête. *ncurses* est stoppé, l'image est sauvegardée et l'interface est supprimée.